Healthcare Hackathon: Repo Investigation & Build Plan

Oct 17, 2026 · @Nilakanta Mutum

Scope and limits

Offline AI triage for community health workers is already a crowded category. A winning entry has to beat it on language and safety design, not on the idea itself.

Constraints used: 24–36 hour build, strong ML + full-stack team, CPU-only laptops, free-tier APIs only, Northeast India focus, judges weighting real-world impact.

What was examined (research date 17 Oct 2026): 14 candidates were checked at README/model-card level or deeper. Only one (SHIFA) was inspected at source-file level (guardrails.py) and through the GitHub API. Another 5 projects (ASHABot, Jugalbandi, TABIB, BridgeDX, UbuntuAfya) were reviewed from articles only, with no code inspected.

Not done: No repository was cloned or executed. Latency figures are from third-party benchmarks, not our hardware. Star counts are missing where a page did not show them. The 20-repo target was not met; depth went to the candidates that fit the constraints.

Prompt changes made: The original brief picked the repo before the problem. This report picks the problem first, then the code that covers most of it.

A. Executive verdict

Start from Mosss-OS/trij (Apache 2.0) as the app shell. Add AI4Bharat's IndicConformer and IndicTrans2 (both MIT) for Manipuri, and write your own deterministic danger-sign engine.

The best technical fit is not the recommended base. Mist-Labs/shifa scores highest on raw merit. It has offline triage, rules the model cannot override, a 60-case validation suite and outbreak clustering. But the GitHub API reports `license: null`, so you have no legal right to fork it. Use it as a reference design only, unless its author grants a license in writing before the hackathon.

Trij wins because it is legally clean and already does the plumbing you would otherwise burn 10 hours on. That covers an offline PWA, patient records, referral PDFs, sync, and a local Ollama bridge to Gemma 4.

It lacks exactly what makes your entry different. Its voice uses the browser Web Speech API, which does not cover Manipuri. It also has no safety rule layer. Those gaps are your contribution.

The one fact that should shape the whole design: the best open Manipuri speech recognizer makes roughly 4 word errors in 10 (SraVaani benchmark, IndicConformer 39.2% WER). A voice-to-decision pipeline that trusts the transcript will mis-triage. Your differentiator is a design that cannot.

B. Best 10 repositories

No single repository covers more than half the build. Each is scored for the role it would play here: app foundation, language component, or reference only.

Rank	Repository	Role	What it does	Stack	Verified activity	Strongest advantage	Main limitation	Score /100
1	Mosss-OS/trij	Foundation	Offline CHW triage PWA: photo/voice triage, patient records, referral PDF, sync	React 19, TypeScript, Dexie, Supabase, WebLLM/Ollama, Gemma 4	146 commits; stars not shown; Gemma 4 Good Hackathon entry	Apache 2.0; Ollama bridge already wired	Web Speech API has no Manipuri; no deterministic safety rules; Supabase dependency	72
2	Mist-Labs/shifa	Reference only	Offline IMCI triage, rules the model cannot override, 60-case validation, outbreak clustering	Expo/React Native, Python, fine-tuned Gemma 4 E2B GGUF, Whisper	75 commits; 0 stars; last push 18 May 2026	Best safety architecture found	**No license** (API: null). Cannot be forked legally	75 raw, fails license gate
3	AI4Bharat/IndicConformerASR	Component	Speech-to-text for 22 Indian languages, Manipuri included	Conformer CTC/RNNT, NeMo; 600M multilingual via transformers + ONNX	12 commits; published Sep 2024	Only open Manipuri ASR found; MIT	About 39% WER on Manipuri	70
4	AI4Bharat/IndicTrans2	Component	Translation across 22 languages, Meitei Mayek and Bengali scripts for Manipuri	Transformer NMT; distilled 200M variants	Repo updated Dec 2024	Native Meitei Mayek script; MIT	CPU speed on our laptops unverified	70
5	microsoft/byoeb	Reference	Expert-in-the-loop WhatsApp bot; base of ASHABot	Flask, MongoDB, ChromaDB, Azure OpenAI, Azure Translator	43 stars, 16 forks, 84 commits, 20 open PRs	Proven clinician-review loop; MIT code	Needs Azure and WhatsApp keys: breaks free-tier and offline constraints	65
6	ai4bharat/indic-parler-tts	Component	Text-to-speech, 20 Indian languages incl. Manipuri	Parler-TTS Mini	Hugging Face model, not a GitHub repo	Manipuri voice output	Slow without GPU; license not checked	60

Rank	Repository	Role	What it does	Stack	Verified activity	Strongest advantage	Main limitation	Score /100
7	medic/cht-core	Reference	Mature offline toolkit used by 24,000+ health workers	CouchDB, PouchDB, TypeScript	523 stars	Real-world danger-sign workflows	AGPL-3.0; steep config; no AI; too heavy for 36 hours	58
8	Teddy-XiongGZ/MedRAG	Optional	Medical RAG toolkit + MIRAGE benchmark (7,663 questions)	BM25, MedCPT, PubMed/StatPearls corpora	Last noted update Apr 2024	Benchmarked retrieval design	English-only; corpora too large for CPU; license not checked	52
9	medplum/medplum	Not recommended	FHIR backend and EHR platform	TypeScript, PostgreSQL, FHIR R4	~2,100 stars (secondary source)	Apache 2.0, standards-based	Wrong layer for a 36-hour demo	50
10	fieldmedic-gemma-4-e2b-lora	Optional	Gemma 4 E2B LoRA for offline triage	Unsloth LoRA	17/20 eval pass (self-reported)	Ready-made triage adapter	420-example dataset not released; English only	45

Score weights follow the brief: implementation 20, innovation potential 20, completeness 15, feasibility 15, impact 10, AI 10, docs 5, license 5. Section C shows the breakdown for the top three.

C. Deep review of the top three

Criterion (weight)	Trij	SHIFA	IndicConformer
Implementation (20)	15	18	15
Innovation potential (20)	12	14	15
Completeness (15)	12	13	8
Hackathon feasibility (15)	11	8	9
Healthcare impact (10)	7	8	9
AI / agentic (10)	6	9	7
Docs (5)	4	5	2
License (5)	5	0	5
Total	**72**	**75 (gated)**	**70**

Trij — the shell you build on

Architecture per its README: TanStack routes for triage, document scan, patients and supervisor views. Core logic sits in `src/lib/` : `gemma.ts` (WebLLM, Ollama and demo engines), `gemma-prompt.ts` , `db.ts` (Dexie/IndexedDB), `sync.ts` , `voice.ts` and `referral.ts` (PDF).

- **Strengths:** offline-first storage and sync already done; an Ollama path means your Python service can sit behind the same interface; Docker compose brings up app + Ollama + local Supabase.
- **Weaknesses:** voice is Web Speech API (no Manipuri); triage is pure LLM output with no rule layer; login depends on Supabase. I read the README and folder layout, not the source files, so code quality is unverified.
- **Prerequisites:** Node 22+, Bun, Docker, Ollama with `gemma4` .
- **Fit:** good. You swap one engine and add one service rather than building an app.

SHIFA — the design to learn from

Its guardrails.py is 172 lines of regex rules with negation detection ("no convulsions" does not fire). The key design choice: overrides read only the original intake text, never the model's own list of danger signs, so a model hallucination cannot trigger an emergency. Copy that principle; it is an idea, not code.

- **Reported results:** 60 synthetic WHO IMCI cases; mobile model at 95.0% decision accuracy and 100% urgent recall. Self-reported on its own synthetic set, so treat as a claim.
- **Weaknesses:** English-only regex; 3.2 GB model; Android/Expo build; African languages and contexts; no license.
- **Fit:** reference only. Fine-tuning Gemma is a GPU job and is not needed for your version.

AI4Bharat language stack — the hard part

IndicConformer's 600M multilingual model loads through `transformers` + ONNX runtime, avoiding the heavy NeMo install. IndicTrans2's distilled 200M models translate Manipuri in Meitei Mayek (`mni_Mtei`) and Bengali script (`mni_Beng`), and need `IndicProcessor` from IndicTransToolkit.

- **Strengths:** the only open stack found that covers Manipuri in all three of ASR, translation and TTS; MIT on ASR and MT.

- **Weaknesses:** Manipuri ASR around 39% WER, measured on just 0.33 hours of test audio, so even that number is thin evidence. Indic Parler-TTS is slow on CPU. No Kuki-Chin language support anywhere.
- **Fit:** essential, but only if the design assumes the transcript is wrong.

D. Existing work and differentiation

Every headline feature you might pitch is already shipped somewhere. What none of them does is Manipuri voice, plus a safety design built for a recognizer that is wrong 4 words in 10.

Project	Language(s)	Offline	Rule layer	Setting
ASHABot (Khushi Baby + Microsoft)	Hindi, English	No (GPT-4, WhatsApp)	Nurse answers what the bot can't	ASHAs, Rajasthan
TABIB (hackathon, Jun 2026)	Hindi, English	No	Multi-agent LLM, 3 urgency levels	ASHAs, rural India
SHIFA	6 African languages	Yes	Deterministic IMCI overrides	Crisis camps, Africa
Trij	7 incl. Hindi	Yes	None found	CHWs, general
FieldMedic	English	Yes	Prompt-level refusals	Disaster, displaced
Jugalbandi	10 Indian languages	No	None stated	Government schemes, not clinical

A. Already commonplace (do not pitch these as new): offline LLM triage, three-level urgency output, WhatsApp delivery, voice input, referral slips, case sync to a dashboard.

B. Existing but improvable: rule overrides exist only in English regex (SHIFA). Outbreak flags exist with geospatial clustering; relief camps are fixed sites, so simple per-camp counts are enough and easier to explain.

C. Differentiated combination (your entry): Manipuri speech in, Manipuri speech out. Danger signs read back to the worker for yes/no confirmation before any rule fires. Rules decide urgency; the LLM only extracts and explains. Searches of GitHub topics and Hugging Face for Manipuri health tools found only a general Meitei Mayek chatbot and NLP utilities, nothing clinical. That supports "uncommon", not "never done".

D. Research direction (state it, don't overclaim): measuring how ASR error rates turn into triage errors, and whether a confirmation loop restores urgent recall. You can produce a small version of this measurement in the hackathon.

Blind spot you must address in the pitch: most displaced people in the Churachandpur and Kangpokpi camps are Kuki-Zo and do not speak Meiteilon as a first language (newsreel.asia). No open speech or translation model covers Kuki-Chin languages. A Manipuri-only tool pitched for "Manipur relief camps" will look one-sided to any judge who knows the conflict. Scope it honestly: Meiteilon first, an English/Hindi tap-through fallback that works for every camp, and Kuki-Chin support named as the gap you need data for.

E. Problem statement

More than 58,000 people displaced by the conflict in Manipur still live in 281 relief camps with minimal healthcare, and residents report recurring measles, dysentery and fever (Amnesty International, May 2025; JURIST). Camp volunteers and ASHAs decide every day whether a sick child or pregnant woman can wait or must travel now, usually without a doctor, often without reliable internet, and in a language that AI health tools do not speak.

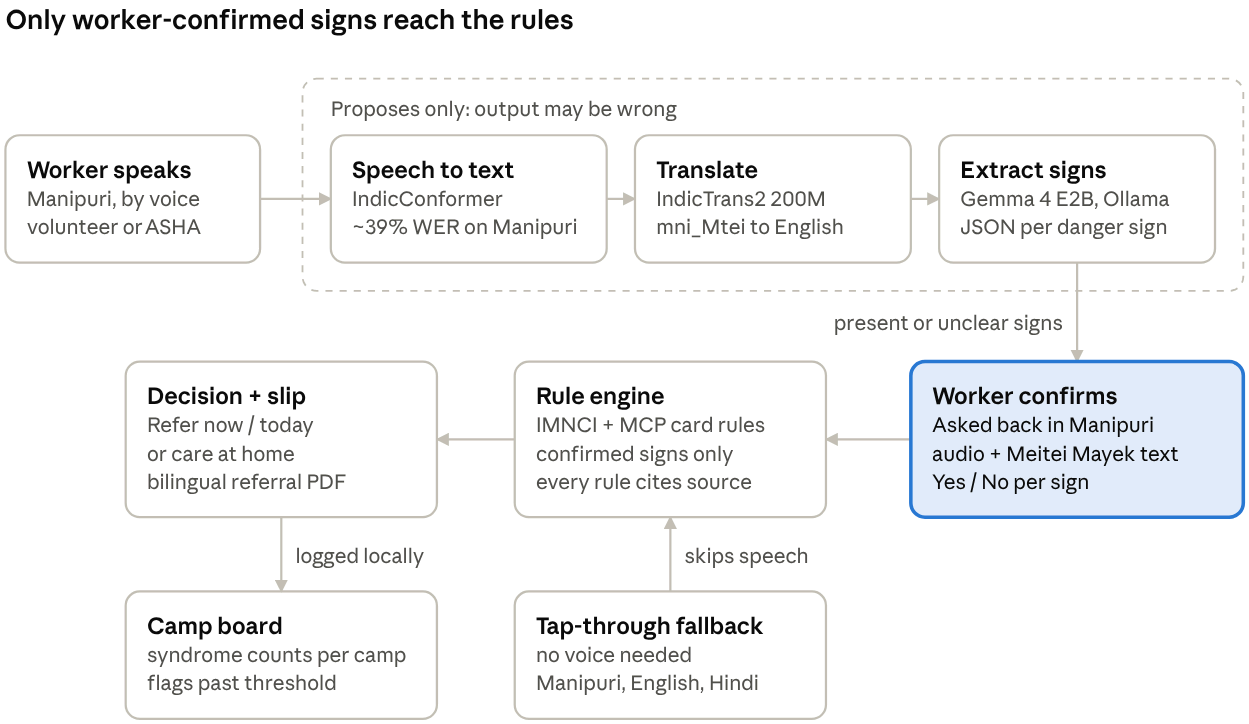
Existing AI triage tools work in Hindi, English or African languages and send transcripts straight to a model. For Manipuri, the best open speech recognizer gets about 4 words in 10 wrong, so a tool that trusts its own transcript will miss emergencies.

We will build a laptop-only, offline assistant that listens in Manipuri, confirms each danger sign back to the worker in Manipuri before deciding, and applies fixed WHO/IMNCI and MCP-card rules that no model output can override. It flags cases for referral; it does not diagnose. We will show that the confirmation step keeps urgent-case recall high even when speech recognition is badly wrong.

F. Project concept

The AI never makes the urgency call. Speech recognition and the LLM only propose danger signs; the worker confirms them; fixed rules decide.

Target users: camp volunteers and ASHAs triaging children under 5 and pregnant or postpartum women. Secondary: the camp coordinator who watches for outbreaks.



architecture · speech path, readback, rules, fallback

Everything inside the dashed box may be wrong. Nothing reaches the rule engine without a worker's tap, and the tap-through path skips speech entirely.

Features

1. **Manipuri voice intake.** Worker describes the case aloud; IndicConformer transcribes, IndicTrans2 translates to English.
2. **Structured extraction.** Gemma 4 E2B (local, via Ollama) fills a fixed JSON schema: age band, pregnancy status, and each danger sign as present / absent / unclear.
3. **Confirm-before-decide readback (core novelty).** Every sign marked present or unclear is asked back in Manipuri, as pre-recorded audio plus Meitei Mayek text, with Yes / No buttons. Unasked urgent signs are prompted as a short checklist.
4. **Deterministic rule engine.** Rules run only on confirmed signs and output Refer now / Refer today / Care at home. Each rule cites its source line (IMNCI chart booklet, MCP card).
5. **Tap-through fallback.** The same checklist works with no voice at all, in Manipuri, English or Hindi. This is what serves non-Meiteilon speakers.

6. **Referral slip + camp board.** Trij's PDF slip, bilingual. A coordinator view counts syndromes per camp (fever with rash, watery diarrhoea) and flags when a configurable threshold is crossed.

Stack

Layer	Choice	Why
Frontend	Trij PWA (React 19, Dexie)	Offline records, sync and PDF already built
Local service	Python FastAPI on localhost	Hosts ASR, MT and rules behind one API
ASR	IndicConformer 600M multilingual, ONNX on CPU	Only open Manipuri ASR
Translation	IndicTrans2 distilled 200M (mni_Mtei ↔ en)	Smallest model with Meitei Mayek
LLM	Gemma 4 E2B via Ollama, JSON-constrained output	Apache 2.0; about 7–10 tokens/s on a laptop CPU (itsfoss test)
Rules	YAML rule file + Python evaluator + unit tests	Auditable, testable, model-independent
Speech out	Indic Parler-TTS, rendered once for ~30 fixed prompts	Avoids slow CPU synthesis at runtime
Storage	Dexie (IndexedDB); Supabase off for the demo	No PHI leaves the laptop

Measurable impact

On a synthetic test set, report three conditions side by side: LLM on raw transcript, rules on raw transcript, rules on confirmed signs. Measure urgent recall, over-referral rate and end-to-end seconds per case. The claim judges can check is the gap between the first and third conditions.

G. Implementation plan (36 hours, 4 people)

The build lives or dies in hour 2: if the three models don't all run on one laptop by then, switch to the tap-through demo and keep going.

Roles assumed: **A** speech/ML, **B** LLM + rules, **C** frontend, **D** evaluation + content and pitch. You need at least one fluent Meiteilon speaker; without one, the readback and test audio can't be checked.

1. **Before the event (if rules allow):** download all model weights; render the ~30 Manipuri prompt audio files on free Colab; collect the IMNCI and MCP card danger-sign lists.
2. **Hours 0–2 — feasibility gate.** Load IndicConformer, IndicTrans2 and Gemma 4 E2B on one laptop; time each on one sample. *Gate:* each runs. *Fallback:* drop live ASR, demo typed Manipuri + tap-through.
3. **Hours 2–10 — parallel foundations.**
   - A: FastAPI `/asr` and `/translate`.
   - B: 20–30 rules in YAML with source citations, plus unit tests.
   - C: fork Trij, run demo mode without Supabase login, add Noto Sans Meetei Mayek font.
   - D: write 60 synthetic vignettes (about a third urgent), get them translated and checked.
4. **Hours 10–20 — the core loop.** B: extraction prompt with JSON schema through Ollama. C: readback screen. A: wire Trij's triage route to the service. *Gate at hour 20:* one Manipuri voice case runs end to end.
5. **Hours 20–28 — evidence.** D + A: record vignettes as audio, run the three-condition evaluation, produce one results table. C: camp board.
6. **Hours 28–34 — demo and pitch.** Freeze code at hour 32. Rehearse the scripted demo twice on the presentation laptop with Wi-Fi off.
7. **Hours 34–36 — buffer.**

Live demo script (3 minutes): a volunteer describes a 2-year-old with fever, rash and a fit yesterday. ASR garbles the fit. The readback asks "Did the child have convulsions?", the worker taps Yes, and the rule fires Refer now with its IMNCI citation. Then show the results table: urgent recall with and without readback.

Cut list if behind: camp board first, then the referral PDF, then live TTS. Never cut the rule engine or the evaluation table.

H. Exact resources

Essential

Resource	Link	Access note
Trij (app shell)	github.com/Mosss-OS/trij	Apache 2.0
IndicConformer ASR	github.com/AI4Bharat/IndicConformerASR	MIT
IndicConformer 600M multilingual weights	huggingface.co/ai4bharat/indic-conformer-600m-multilingual	Loads with `trust_remote_code=True`
IndicTrans2 code	github.com/AI4Bharat/IndicTrans2	MIT
IndicTrans2 Indic→En distilled 200M	huggingface.co/ai4bharat/indictrans2-indic-en-dist-200M	Gated: accept terms on Hugging Face first
IndicTrans2 En→Indic distilled 200M	huggingface.co/ai4bharat/indictrans2-en-indic-dist-200M	Needs IndicProcessor
IndicTransToolkit (IndicProcessor)	github.com/VarunGumma/IndicTransToolkit	Required preprocessing
Gemma 4 E2B	via Ollama (`ollama pull gemma4` , per Trij README)	Apache 2.0 since Apr 2026 (the-decoder)
Clinical rule sources	WHO/MoHFW IMNCI chart booklet; MoHFW Mother and Child Protection card	Not opened in this research — fetch the official PDFs and cite page numbers in the rule file

Optional

Resource	Link	Use
Indic Parler-TTS	huggingface.co/ai4bharat/indic-parler-tts	Pre-render Manipuri prompts; check license before use
SHIFA guardrails (reference)	guardrails.py	Read for design; do not copy code (no license)
BYOeB	github.com/microsoft/byoeb	Future clinician-review loop
Bhashini API	Bhashini API docs	Online fallback for ASR/TTS; needs free registration and internet

Resource	Link	Use
Meitei Senter	pypi.org/project/meitei-senter	Sentence splitting for Meitei Mayek, MIT
Manipuri speech corpora	LDC-IL Manipuri resources	Test audio if you lack native recordings

I. Final decision

1. **Which repository to start with?** Mosss-OS/trij.
2. **Why it beats the alternatives here:** it is the only offline CHW triage app found with a permissive license and a local-model bridge. SHIFA is better engineered but unlicensed. BYOeB needs Azure and WhatsApp. CHT is AGPL and too heavy for 36 hours.
3. **What to reuse:** Trij's offline storage, sync, patient records, referral PDF, Ollama engine and PWA shell; AI4Bharat's ASR and translation models as-is; Gemma 4 E2B as-is.
4. **What you must build:** the FastAPI language service, the extraction schema and prompt, the readback screen, the rule engine with cited rules and tests, the evaluation harness and test set, the camp board.
5. **Strongest differentiator:** confirm-before-decide in Manipuri, with evidence that it holds urgent recall up despite bad speech recognition.
6. **Three biggest risks:**
   - *Readback fatigue.* At about 39% WER, extraction will mark many signs "unclear", and long question lists slow real triage. Cap readback at the urgent signs and measure questions per case.
   - *CPU latency.* Gemma 4 E2B produced 6.9 tokens/s in one laptop test, so a long JSON answer can take 20+ seconds. Keep the schema short, keep the model loaded, and show progress on screen.
   - *Credibility.* Rules written by students, no clinician sign-off, and a single-language tool in a two-community conflict. Cite every rule to its source line, get one doctor or nurse to review the rule file before judging if you can, and present the fallback for non-Meiteilon speakers up front.
7. **First concrete step:** today, on the weakest laptop your team will use, load the three models and time one Manipuri sentence through ASR → translation → Gemma JSON. That number decides whether live voice is in the demo.

Sources

Repositories and model cards are linked in the tables above. Context sources opened or read in search results:

- SraVaani 1.0 ASR benchmark (arXiv 2608.08235) — Manipuri WER figures
- Amnesty International, May 2025 — camp counts and conditions
- JURIST — measles, dysentery and fever outbreaks in camps
- newsreel.asia, Jan 2025 — Kuki-Zo camp populations
- Newslaundry, Dec 2025 — camp health conditions
- Microsoft Research on ASHABot and ASHABot paper (arXiv 2409.10913)
- TABIB hackathon entry
- itsfoss CPU LLM benchmark — Gemma 4 E2B speed
- Gemma 4 Apache 2.0 release coverage
- SHIFA repository metadata (GitHub API) — license null